Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

Programa de Licenciatura en Ingeniería en Computadores

Curso: CE-4301 Arquitectura de Computadores I
Curso: CE-1108 Compiladores e Intérpretes



Especificación Proyecto Grupal II

Análisis de Impacto de Memoria Caché y Optimizaciones de Compilador en la Organización de Procesador Propio tipo VLIW

Profesores:
Dr.-Ing. Jeferson González Gómez (CE4301)
MSc. Jason Leitón Jiménez (CE1108)

Grupos: 4 estudiantes
II Semestre 2026

Proyecto Grupal II – Análisis de Impacto de Memoria Caché y Optimizaciones de Compilador en Procesador VLIW Propio

1. Objetivo

Mediante el desarrollo de este proyecto conjunto entre CE4301 (Arquitectura de Computadores I) y CE1108 (Compiladores e Intérpretes), el estudiante aplicará conceptos avanzados de compiladores y arquitectura de computadores para evaluar el impacto de la memoria caché de dos niveles y las optimizaciones de compilador en el rendimiento del procesadores modernos. El proyecto involucra la extensión de la arquitectura VLIW propia implementada en el Proyecto Grupal I –que ya incluye organización segmentada (pipeline) y bundles de instrucciones multi-slot– para soportar una jerarquía de memoria caché de dos niveles, así como el análisis de métricas de rendimiento bajo diferentes configuraciones de optimización de compilador.

2. Descripción General

El rendimiento de los sistemas computacionales modernos depende críticamente de la interacción entre la arquitectura del procesador, la jerarquía de memoria y las optimizaciones aplicadas por el compilador. La memoria caché permite reducir significativamente la latencia promedio de acceso a memoria, mientras que las optimizaciones de compilador pueden mejorar la localidad de referencias, aprovechar opciones de paralelismo mediante reordenamiento, y reducir el número total de instrucciones ejecutadas.
Este proyecto explora estas interacciones mediante la extensión del procesador VLIW propio (desarrollado en el Proyecto Grupal I) con una jerarquía de caché de dos niveles (L1 y L2), y la evaluación sistemática del impacto de diferentes tipos de optimización del compilador propio sobre métricas clave de rendimiento como:

- Tasas de hit/miss de caché L1 y L2
- Ciclos totales de ejecución
- Accesos totales a memoria
- Throughput de instrucciones

La implementación del procesador y su jerarquía de memoria se realizará completamente en SystemVerilog para permitir simulación detallada y medición precisa de todas las métricas de interés, extendiendo la arquitectura VLIW específica desarrollada en el Proyecto Grupal I.

3. Especificación del Sistema

Esta sección detalla los componentes arquitectónicos que deben implementarse para evaluar el impacto de la jerarquía de memoria caché en el rendimiento del procesador VLIW propio. El diseño se enfoca en mantener la compatibilidad con el ISA VLIW (formato de bundle/slots y codificación binaria) desarrollado previamente, mientras se extiende la microarquitectura con un sistema de memoria jerárquico.

3.1. Procesador VLIW Propio Extendido con Jerarquía de Caché

El proyecto extiende el procesador VLIW propio (bundles de al menos 4 slots, con unidades funcionales ALU, LSU, BRU y la unidad criptográfica Feistel4 con su bóveda de llaves) desarrollado en el Proyecto Grupal I, incorporando las siguientes mejoras arquitectónicas obligatorias.

3.1.1. Extensión del Pipeline VLIW con Acceso a Caché

El Proyecto Grupal I ya exige una organización segmentada (pipeline) con, como mínimo, las etapas Fetch (IF), Decode/Dispatch (ID), Execute (EX) y Writeback (WB), en las que el bundle completo (todos sus slots) avanza de forma conjunta por el *datapath*. Para este proyecto, dicho pipeline se extiende con el acceso a la jerarquía de caché L1-D/L2 para las operaciones del slot LSU, ya sea dentro de la etapa EX existente o mediante una etapa adicional dedicada (MEM), a elección y justificación de cada grupo:

- **IF:** Búsqueda del bundle completo desde memoria ROM (acceso en 1 ciclo, sin stalls)
- **ID/Dispatch:** Decodificación de los slots del bundle y despacho hacia sus unidades funcionales
- **EX:** Ejecución en paralelo de las operaciones de cada slot (ALU, BRU, unidad criptográfica) y cálculo de direcciones para el slot LSU
- **MEM (opcional como etapa separada):** Acceso del slot LSU a la caché L1-D para loads/stores
- **WB:** Escritura de resultados en banco de registros y, cuando corresponda, en la bóveda de llaves

El pipeline debe incluir:

- Stalls por accesos a memoria caché desde el slot LSU (L1 miss → acceso L2, L2 miss → memoria). **Importante:** dado que todos los slots de un bundle avanzan en *lockstep* por las etapas del pipeline (sección de Arquitectura VLIW del Proyecto Grupal I), un miss de caché en el slot LSU detiene el **bundle completo** (todos sus slots), no solamente dicho slot.
- Contadores de rendimiento por etapa para medición de métricas
- Interfaz con jerarquía de caché que permita stalls dinámicos del pipeline

Nota sobre hazards: Los stalls por miss de caché son un riesgo **estructural** (contención de un recurso compartido), explícitamente permitido por el diseño VLIW del Proyecto Grupal I, y no implican que el hardware deba detectar o resolver hazards de datos o de control de forma dinámica. Al igual que en el Proyecto Grupal I, el hardware **no debe** implementar *forwarding* ni *scoreboarding* entre bundles: la calendarización estática de las operaciones dentro de los bundles (incluyendo el manejo de riesgos RAW y la inserción de NOPs) sigue siendo responsabilidad exclusiva de quien genera el código (ensamblador propio o compilador de CE1108), tal como se definió en el Proyecto Grupal I.

3.1.2. Extensión de la ISA VLIW propia

El procesador debe mantener exactamente la misma ISA VLIW (formato de bundle/slots y codificación binaria de las instrucciones ya definidas) desarrollada en el Proyecto Grupal I, incluyendo:

- **Instrucciones de las unidades ALU, LSU y BRU:** Operaciones aritméticas, lógicas, acceso a memoria y control de flujo
- **Instrucciones de seguridad:** Cifrado/descifrado Feistel4 (unidad criptográfica), manejo de bóveda de llaves y autenticación/autorización de acceso

Adicionalmente, cada grupo puede (de forma **opcional**) definir **nuevas** instrucciones de control de caché y medición de rendimiento, utilizando codificaciones (*opcodes*) que no estuvieran ya asignadas en el Proyecto Grupal I. Esta es la **única** extensión permitida al ISA; de ejercerla, el grupo debe documentarla explícitamente en `docs/` (codificación binaria, slot que la admite, semántica) y comunicarla a su contraparte de CE1108, dado su impacto en el contrato de interfaz ISA–Compilador.
Nota importante: Para todas las instrucciones ya definidas en el Proyecto Grupal I (ALU, LSU, BRU, unidad criptográfica, bóveda de llaves), cada grupo debe mantener exactamente el mismo formato de bundle/slots, la misma codificación binaria y los mismos registros; no se permite modificarlas. La única extensión admitida es la adición opcional de nuevas instrucciones de caché descrita arriba. Fuera de eso, solo se extiende la microarquitectura con la jerarquía de memoria caché (y, de ser necesario, con una etapa adicional del pipeline dedicada a dicho acceso).

3.2. Jerarquía de Memoria

El sistema de memoria combina una ROM de instrucciones de acceso directo con una jerarquía de caché de datos de dos niveles (L1-D y L2). Esta simplificación es adecuada dado que el proyecto ejecuta una sola aplicación a la vez, eliminando la necesidad de una caché de instrucciones y reduciendo la complejidad de implementación sin sacrificar el análisis del comportamiento de la caché de datos.

3.2.1. ROM de Instrucciones

La etapa IF accede a una memoria ROM síncrona, organizada en bundles según el ancho de bundle que cada grupo definió en el Proyecto Grupal I (mínimo 4 slots), que contiene el programa compilado. Al ejecutar una sola aplicación estática, el acceso es siempre en 1 ciclo sin posibilidad de miss, por lo que no genera stalls en el pipeline.

- Tamaño máximo: 64 KB. El número exacto de bundles almacenables depende del ancho de bundle elegido por cada grupo; por ejemplo, con un bundle de 128 bits (4 slots de 32 bits), 64 KB equivalen a 4096 bundles (16384 palabras de 32 bits).
- Acceso: síncrono, 1 bundle completo por ciclo
- Inicializada en simulación con el binario del benchmark a ejecutar

3.2.2. Caché L1-D (Datos)

- Tamaño: 4 KB (1024 palabras de 32 bits, simulable en Icarus Verilog)
- Asociatividad: 2-way set associative
- Tamaño de línea: 32 bytes (8 palabras)
- Política de escritura: **A definir por cada grupo** (write-through o write-back; sección 7.1.3)
- Política de asignación en miss de escritura: **A definir por cada grupo** (write-allocate o no-write-allocate; sección 7.1.3)
- Política de reemplazo: **A definir por cada grupo** (sección 7.1.3)
- **Write buffer obligatorio** hacia L2 (sección 3.4), independientemente de la política de escritura elegida
- Hit time: 1 ciclo

3.2.3. Caché L2 (Nivel 2)

Caché L2 Unificada:

- Tamaño: 16 KB (4096 palabras de 32 bits, simulable en Icarus Verilog)
- Asociatividad: 4-way set associative
- Tamaño de línea: 32 bytes (8 palabras)
- Política de escritura: **write-back, obligatoria para todos los grupos** dado el alto costo de latencia hacia memoria principal (sección 7.1.4)
- Política de reemplazo: **A definir por cada grupo** (sección 7.1.4)
- **Eviction buffer obligatorio** hacia memoria principal (sección 3.4)
- Hit time: 8 ciclos

3.2.4. Política de Inclusión L1/L2

Además de las políticas de escritura y reemplazo, cada grupo debe definir y documentar la **política de inclusión** entre L1-D y L2, siguiendo los conceptos de la Lección 14b (*Caché en Procesadores Modernos*):

- **Inclusiva** ($L1 \subseteq L2$): todo bloque presente en L1-D debe estar garantizado también en L2. Esto obliga a implementar **invalidación hacia atrás** (*backward invalidation*): cuando L2 desaloja un bloque por capacidad y ese bloque está presente en L1-D, L2 debe invalidarlo también en L1-D. Es la opción más exigente de implementar, pero permite practicar directamente el mecanismo visto en la Lección 14b.

- **No inclusiva** (NINE, *not inclusive, not exclusive*): no se impone ninguna relación de contenido entre L1-D y L2; un bloque puede o no estar duplicado en ambos niveles según el patrón de accesos. Es la opción de implementación más simple, y la que debe documentarse explícitamente si se elige (no basta con omitir el tema).

Dado que este proyecto implementa un único núcleo (sin coherencia multiprocesador), la ventaja clásica de la inclusión — simplificar el filtro de *snooping* entre varias cachés L1 privadas — no aplica funcionalmente aquí; la elección entre inclusiva y NINE es, en este contexto, principalmente una decisión de diseño con valor pedagógico, y debe justificarse como tal en la documentación.
Opción avanzada (crédito extra, opcional): un grupo puede optar por una política **exclusiva** ($L1 \cap L2 = \emptyset$) en vez de inclusiva o NINE. Esto requiere implementar la **migración** de bloques entre niveles (un acierto en L2 ante un desacierto en L1-D mueve el bloque en vez de copiarlo, y un desalojo de L1-D hacia L2 en vez de descartarse), de forma análoga a la caché víctima. Esta opción es estrictamente más compleja que inclusiva o NINE y debe acordarse con el profesor antes de la Iteración 4.
Cada grupo debe seleccionar **una** de las dos opciones obligatorias (inclusiva o NINE) a más tardar en la Iteración 4 (sección 7.1.4), documentar su elección en `docs/` con la justificación correspondiente, y mantenerla consistente con el resto del diseño de la jerarquía (política de reemplazo de L2, manejo de desalojos, y comportamiento de los write buffers de la sección 3.4).

3.2.5. Memoria Principal

El modelo de memoria principal debe simular realistamente las latencias de acceso a memoria externa, operando en un dominio de reloj separado para reproducir la diferencia de velocidad entre procesador y memoria.
Especificaciones del modelo de memoria:

- **Frecuencia de operación:** 20–50 MHz (vs 100 MHz del procesador), generada por divisor de frecuencia o PLL; cada grupo elige y documenta su valor dentro de ese rango
- **Latencia de primera palabra:** 25 ciclos del procesador, medidos desde que se emite la solicitud hasta que llega la primera palabra de la línea
- **Transfer rate del burst:** 1 palabra por ciclo de **memoria** (no de procesador); una línea completa de 32 bytes (8 palabras) requiere 7 ciclos de memoria adicionales para las 7 palabras restantes después de la primera
- **Tamaño:** 64 KB (16384 palabras, límite simulable en Icarus Verilog)
- **Interfaz:** Burst mode para transferencia eficiente de líneas completas de caché

Cálculo de la penalidad total de un miss hacia memoria principal: dado que la memoria opera en un dominio de reloj más lento que el procesador, los 7 ciclos de memoria restantes del burst deben convertirse a ciclos de procesador según la razón de frecuencias elegida por el grupo:

$$T_{\text{mem}} = 25 + 7 \times \left\lceil \frac{100\,\text{MHz}}{f_{\text{mem}}} \right\rceil \quad \text{[ciclos de procesador]}$$

Por ejemplo, con $f_{\text{mem}} = 25\,\text{MHz}$ (razón 4×): $T_{\text{mem}} = 25 + 7 \times 4 = 53$ ciclos de procesador. Cada grupo debe calcular y documentar su propio valor de T_{mem} según la frecuencia de memoria que elija, y usarlo consistentemente en los contadores de ciclos perdidos (sección 5).
Implementación realista: El controlador de memoria debe manejar crossing de dominios de reloj, implementar timeouts realistas, y generar los stalls prolongados que experimentaría un procesador real. Esta implementación permite medir el impacto real de las latencias de memoria y justifica la necesidad de una jerarquía de caché efectiva.

3.3. Integración Caché-Procesador

El pipeline del procesador debe manejar stalls cuando ocurren misses en la jerarquía de caché, originados por el slot LSU de un bundle. Dado que todos los slots de un bundle avanzan juntos por las etapas del pipeline, estos stalls detienen el **bundle completo**, no solo el slot LSU:
Comportamiento en accesos a memoria:

- **L1 hit:** El pipeline continúa sin stalls (1 ciclo)
- **L1 miss, L2 hit:** Bundle completo en stall por 8 ciclos adicionales (L2 hit time)
- **L1 miss, L2 miss:** Bundle completo en stall por $8 + T_{\text{mem}}$ ciclos adicionales (8 ciclos para determinar el miss en L2, más la penalidad de memoria principal T_{mem} definida arriba, que ya incluye el burst completo de la línea)

Implementación de stalls:

- Señales de control `cache_stall` desde jerarquía de memoria
- Congelamiento del PC y de los registros de segmentación del pipeline (afectando todos los slots del bundle en vuelo) durante stalls
- Contadores de ciclos perdidos por tipo de miss para métricas
- Interfaz simple entre cache controller y pipeline control unit

Nota: El pipeline interno de las cachés es opcional y queda como mejora avanzada. La implementación básica puede usar acceso combinacional con stalls apropiados.

3.4. Buffers de Escritura Obligatorios

A diferencia de las políticas de escritura y reemplazo, el uso de **buffers de escritura** no es una decisión opcional: todo grupo debe implementar los dos buffers siguientes, siguiendo los conceptos de la Lección 14b (*Non-Blocking Cache y Write Buffers*):

- **Write buffer L1-D → L2:** FIFO que recibe las escrituras que salen de L1-D hacia L2 (en write-through, cada store; en write-back, las líneas sucias desalojadas) y las drena de forma asíncrona, para que el pipeline no se detenga en cada escritura individual. Tamaño: a elección del grupo, 2–8 entradas.
- **Eviction buffer L2 → memoria principal:** FIFO que recibe las líneas sucias desalojadas de L2 y las drena de forma asíncrona hacia memoria principal, para que el *fill* de la línea entrante no tenga que esperar a que termine el desalojo. Tamaño: a elección del grupo, 4–16 entradas.

Ambos buffers deben manejar el riesgo **RAW en memoria** (*Read After Write*): si una dirección presente en el buffer es leída antes de drenar, el grupo debe elegir e implementar una de las dos estrategias siguientes, y documentar su elección:

- **Stall:** esperar a que el buffer drene la entrada en conflicto antes de completar la lectura.
- **Store-to-load forwarding** (preferido en procesadores modernos): reenviar el dato directamente desde el buffer hacia la lectura, sin esperar el drenado.

El tamaño de cada buffer, la estrategia de drenado, y el mecanismo elegido para el riesgo RAW en memoria son parámetros configurables por el grupo, pero la **presencia** de ambos buffers es un requisito funcional obligatorio, independiente de las políticas de escritura, reemplazo e inclusión elegidas en las secciones anteriores.

4. Compilador y Optimizaciones

De parte del compilador se debe desarrollar los siguientes componentes:

1. Parser y AST/IR (30 pts)
   - Lexer y parser que acepten la sintaxis definida en la plantilla, así como el enlazado entre archivos. El cual debe de corregir los aspectos del primer proyecto, tanto a nivel sintáctico como semántico. Cabe destacar la implementación de las tablas es libre de cada grupo, sin embargo, debe ser implementadas de manera eficiente.
   - Generación de una representación intermedia (IR) de tres direcciones. Esta representación será definida por cada grupo y deberá contemplar que sea útil para las optimaciones de código, se debe de añadir una bandera para generar el código intermedio, de manera similar al ensamblador.
2. Transformaciones de optimización: El compilador será capaz de mejorar el código con el fin de aumentar el rendimiento general del sistema. Se solicita que se implemente las siguientes técnicas:
   - Loop unrolling con factor configurable y heurística.
   - Eliminación de código muerto (DCE). (Esto generalmente se realiza en la representación intermedia o en el ensamblador)
   - Reordenamiento de instrucciones bajo condiciones de seguridad (preservar dependencias de datos y control).
   - Vaciado de registros, Spilling register.
   - **Calendarización estática y empaquetado de bundles:** El compilador debe asignar las operaciones generadas a los slots del bundle VLIW definido en el Proyecto Grupal I, respetando su ancho, los tipos de unidad funcional de cada slot, y las dependencias de datos y de control (sin asumir *forwarding* ni *scoreboarding* de hardware entre bundles). Debe insertar NOPs en los slots no utilizados y producir binario (o ensamblador propio, traducible por el ensamblador de CE4301) directamente ejecutable sobre el procesador VLIW, conforme al contrato de interfaz ISA–Compilador definido en el Proyecto Grupal I. Se debe agregar una bandera para ver la calendarización de cada paquete.

3. Simulador de ejecución y renombramiento (se utilizará el modelo VLIW de calendarización estática que se desarrolle en arquitectura. Cada grupo debe proporcionar simulaciones por ciclos para revisar esta parte)
    - Cada grupo debe de proporcionar un conjunto de pruebas (código representativo) para verificar el funcionamiento de estos módulos. (En la defensa se proporcionará un par más).
4. Prototipo de IDE. Se requiere que cada grupo implemente un IDE que pueda realizar lo siguiente:
    - Sugerencias de código cuando se está escribiendo las instrucciones.
    - Correcciones de errores automáticos (falta de terminación de línea, llaves o paréntesis).
    - Detección de errores marcados gráficamente y con buena descripción del error (Puede ser subrayado o cualquier otra manera).
    - Detección de clones de código Tipo I y Tipo II. (Esto debe de ser capaz entre archivos)

5. Métricas de Evaluación

La evaluación del rendimiento del sistema requiere la medición precisa de múltiples métricas que permitan analizar el impacto de la jerarquía de caché y las optimizaciones del compilador. El sistema debe implementar contadores de hardware integrados que capturen información detallada sobre el comportamiento del procesador y el sistema de memoria durante la ejecución de diferentes benchmarks.

5.1. Métricas del Procesador

Los contadores del procesador cuantifican el comportamiento del pipeline VLIW y el costo de los stalls provocados por la jerarquía de memoria y por los saltos. Estas métricas establecen la línea base para comparar configuraciones y analizar el impacto de las optimizaciones del compilador. Dado que cada bundle agrupa varias operaciones de slots distintos, se distingue entre **bundles ejecutados** (uno por ciclo en ausencia de stalls) e **instrucciones ejecutadas** (la suma de operaciones no-NOP en todos los slots de todos los bundles), siendo esta última la que refleja el paralelismo a nivel de instrucción efectivamente explotado.

Cuadro 1: Métricas del procesador VLIW segmentado

Métrica	**Definición**
Ciclos totales de ejecución	Ciclos desde inicio hasta fin de ejecución
Bundles ejecutados	Total de bundles completados por el pipeline
Instrucciones ejecutadas	Total de operaciones no-NOP completadas en todos los slots
IPC	Instrucciones ejecutadas / Ciclos totales
Stalls por misses L1-D	Ciclos perdidos por bundles detenidos en el slot LSU (loads/stores)
Stalls por saltos	Ciclos perdidos por vaciado del pipeline en saltos (branch flush) o slots

5.2. Métricas de la Jerarquía de Caché

Cada nivel de la jerarquía expone contadores que permiten calcular el AMAT (*Average Memory Access Time*) y correlacionar los fallos de caché con la degradación del IPC. La Tabla 2 detalla las métricas mínimas que deben medirse en cada nivel; el hit rate y el miss rate se derivan de los contadores de accesos, hits y misses registrados directamente por hardware.

Cuadro 2: Métricas por nivel de caché

Métrica	**L1-D**	**L2**
Accesos totales	reads y writes por separado	desde L1-D
Hits / Misses	desglosados por read y write	✓
Hit rate	✓	✓
Miss rate	✓	✓

Si el grupo elige la política de inclusión **inclusiva** (sección 3.2.4), debe además contar las **invalidaciones hacia atrás** ejecutadas sobre L1-D, para poder cuantificar su costo en el hit rate de L1-D.

5.3. Métricas de Memoria Principal

El modelo de memoria principal registra tres métricas orientadas a cuantificar la presión que ejerce la jerarquía de caché sobre el bus de memoria: el **número total de accesos** (cada uno originado por un miss en L2), el **total de ciclos consumidos en transferencias** (incluyendo la latencia de burst completo de línea, es decir T_{mem} por cada acceso), y la **utilización del ancho de banda**, expresada como fracción del ancho de banda teórico máximo del modelo.

6. Análisis de Optimizaciones de Compilador

Esta sección define las optimizaciones de compilador que deben documentarse y validarse, las pruebas mínimas obligatorias que debe construir cada grupo, y las métricas que se deben registrar para cada prueba.

Optimizaciones requeridas

El compilador propio debe implementar las siguientes optimizaciones sobre la representación intermedia o el ensamblador generado. Para cada una, el grupo debe documentar la regla, las precondiciones, la transformación aplicada y ejemplos concretos:

- Loop unrolling (factor configurable; parcial/total).
- Eliminación de código muerto (DCE) usando análisis de liveness.
- Reordenamiento de instrucciones seguro dentro de bloques básicos (respetando RAW/WAR/WAW y dependencias de control).
- Spilling register.

Pruebas obligatorias

Cada grupo debe construir un conjunto mínimo de programas de prueba que ejerciten individualmente cada una de las optimizaciones anteriores, de forma que su efecto pueda observarse y medirse de manera aislada. Debe incluirse al menos:

- 3 programas para loop unrolling (N constante, N variable pequeña, N grande).
- 3 programas con dependencias que exijan renombramiento (WAW/WAR).
- 3 programas con código muerto mezclado con llamadas con efectos.
- 3 programas con saltos condicionales y funciones (para reordenamiento y seguridad).
- 3 programas para verificar el Spilling register.

Métricas y medidas

Con el fin de cuantificar el efecto de cada optimización sobre las pruebas anteriores, cada grupo debe registrar, de forma gráfica (puede estar integrada en el mismo IDE), las siguientes métricas para cada prueba:

- Conteo de instrucciones (antes/después).
- Tamaño de código (bytes).
- Ciclos estimados por el simulador.
- Tiempo de compilación.
- Número de instrucciones eliminadas / reordenadas / renombradas.

6.1. Benchmarks de Evaluación

Cada grupo deberá plantear las pruebas correspondientes para demostrar el funcionamiento del compilador, sin embargo, el día antes de la defensa el profesor proporcionará algunos ejemplos que podrían ser utilizados para probar el funcionamiento.

7. Metodología de Desarrollo

El desarrollo del proyecto debe seguir una metodología iterativa e incremental que permita validar cada componente antes de proceder al siguiente nivel de complejidad. Esta aproximación es fundamental dado que el proyecto integra múltiples subsistemas complejos (procesador, jerarquía de caché, compilador) que requieren validación independiente y posterior integración.

7.1. Arquitectura del Procesador (CE4301)

Las iteraciones de arquitectura construyen el sistema de hardware de forma incremental. En las etapas donde aplica, se identifican las **decisiones de diseño** que cada grupo debe tomar, documentar y justificar considerando: complejidad de implementación en SystemVerilog, impacto esperado en rendimiento para su conjunto de benchmarks, trade-offs entre área y rendimiento, e interacción con las optimizaciones del compilador.

7.1.1. Iteración 1: Línea Base del Procesador VLIW

Esta iteración establece la línea base del sistema retomando el procesador VLIW propio (bundle de slots, pipeline de Fetch/Decode-Dispatch/Execute/Writeback) desarrollado en el Proyecto Grupal I, validando que su organización segmentada y el despacho en paralelo de los slots del bundle hacia sus unidades funcionales (ALU, LSU, BRU, unidad criptográfica) funcionen correctamente antes de agregar la complejidad de la jerarquía de memoria. Dado que el hardware VLIW no realiza detección ni resolución dinámica de hazards (sección de Arquitectura VLIW del Proyecto Grupal I), esta iteración no introduce manejo de hazards nuevo: se limita a confirmar que la calendarización estática ya definida sigue funcionando sobre el pipeline existente. La validación incluye testbenches simples que ejecuten bundles representativos del ISA propio y la implementación de contadores de rendimiento básicos (ciclos, bundles ejecutados, instrucciones ejecutadas) que servirán de base para las métricas más avanzadas en iteraciones posteriores.

7.1.2. Iteración 2: Modelo Realista de Memoria Principal

Esta iteración implementa un modelo realista de memoria principal que simule las latencias de acceso a memoria externa. Se debe diseñar un controlador que opere a frecuencia menor que el procesador (20–50 MHz vs. 100 MHz) mediante divisores de frecuencia o PLLs, simule la latencia T_{mem} definida en la sección 3.2.5 (que incluye tanto la latencia de primera palabra como el burst completo de la línea), e implemente el eviction buffer obligatorio de la sección 3.4. Esta iteración es crítica porque establece la baseline de rendimiento sin caché, permitiendo cuantificar posteriormente el beneficio real de la jerarquía de memoria.

7.1.3. Iteración 3: Caché L1

En esta etapa se diseña e implementa la caché L1-D. La integración con el pipeline requiere especial atención al manejo de stalls por misses en el slot LSU, implementando la lógica de control que pause correctamente el **bundle completo** (todos sus slots, no solo el LSU) en las etapas del pipeline sin corromper el estado. Se deben desarrollar testbenches específicos para validar la coherencia interna y comenzar la medición de métricas (hit rates, miss penalties, stalls del pipeline).

Decisiones de diseño (L1-D): Cada grupo debe seleccionar y documentar:

- **Política de escritura:** write-through (toda escritura se propaga inmediatamente al write buffer hacia L2; coherencia más simple pero mayor tráfico hacia L2) o write-back (las escrituras acumulan en L1 y se propagan solo en evictions, a través del mismo write buffer; menor tráfico pero requiere dirty bits y control más complejo).
- **Política de asignación en misses de escritura:** write-allocate (traer la línea antes de escribir, combinación típica con write-back) o no-write-allocate (escribir directamente al nivel siguiente, combinación típica con write-through).
- **Política de reemplazo:** LRU (1 bit por set), Random (LFSR) o FIFO.
- **Mecanismo de riesgo RAW en memoria** para el write buffer L1-D→L2 (stall o store-to-load forwarding; sección 3.4). El uso del buffer en sí es obligatorio; solo su tamaño (2–8 entradas) y el mecanismo de RAW quedan a elección del grupo.

7.1.4. Iteración 4: Caché L2 y Jerarquía Completa

Esta iteración completa la jerarquía implementando la caché L2 unificada. El aspecto más crítico es el protocolo de coherencia entre L1 y L2 según la política de inclusión elegida (sección 3.2.4), asegurando que actualizaciones, desalojos y (si aplica) invalidaciones hacia atrás se manejen sin pérdida de datos. La interfaz con el modelo de memoria principal de la Iteración 2 permite validar el comportamiento completo con latencias realistas. Los contadores de métricas se extienden para capturar el tráfico entre niveles y el efecto de las políticas de escritura e inclusión elegidas.
Decisiones de diseño (L2): La política de escritura de L2 es write-back para todos los grupos (sección 3.2.4). Cada grupo debe seleccionar:

- **Política de reemplazo:** Pseudo-LRU (tree-based o bit-based), Random, o LFU aproximado con contadores de 2–3 bits.
- **Política de inclusión L1/L2:** inclusiva o no inclusiva (NINE), según la sección 3.2.4; la variante exclusiva (opcional, crédito extra) debe acordarse con el profesor antes de iniciar esta iteración.
- El eviction buffer de la sección 3.4 es obligatorio; solo su tamaño (4–16 entradas) queda a elección del grupo.
- **Non-blocking caché** (opcional, para grupos avanzados): permite múltiples requests en flight (1–4) mientras se atiende un miss.

7.1.5. Iteración 5: Análisis de Rendimiento

La iteración final evalúa exhaustivamente el sistema completo con benchmarks de distintos patrones de acceso a memoria (secuencial, aleatorio, stride). Se documentan y justifican las decisiones de diseño tomadas en las iteraciones anteriores correlacionándolas con los resultados obtenidos: hit rates, IPC, stalls y AMAT. Esta iteración requiere coordinación estrecha con el equipo de CE1108 para correr los benchmarks bajo diferentes niveles de optimización del compilador propio y analizar cómo las decisiones arquitecturales interactúan con el código generado.

7.2. Compilador (CE1108)

El desarrollo del compilador propio también sigue un esquema incremental, coordinado en el tiempo con las iteraciones de arquitectura para permitir la integración en la Iteración 5:

1. Iteración 1: Se debe realizar el IDE con todos sus requerimientos, incluyendo las mejoras del proyecto 1.
2. Iteración 2: Se debe generar la representación intermedia e identificación de bloques básicos.
3. Iteración 3: Se debe implementar el loop unrolling y spilling register. Además, se debe implementar eliminación de código muerto y reordenamiento.

7.3. Integración Arquitectura–Compilador

La integración de ambos componentes constituye el núcleo del análisis experimental del proyecto. Una vez que la jerarquía de caché esté validada (Iteración 4) y el compilador cuente con al menos un nivel de optimización funcional, el equipo debe ejecutar los benchmarks acordados compilando con diferentes niveles de optimización y midiendo el efecto en las métricas de hardware. Esta fase requiere coordinación estrecha entre los equipos de CE4301 y CE1108: el binario/ensamblado generado por el compilador debe ejecutarse correctamente en el procesador VLIW propio (respetando el formato de bundles/slots definido en el Proyecto Grupal I), y las métricas capturadas por los contadores de hardware deben retroalimentar el análisis de efectividad de cada optimización.

8. Entregables

Los entregables del proyecto se organizan por curso, reflejando la evaluación independiente de cada materia.

8.1. Entregables - CE4301 Arquitectura de Computadores I

8.1.1. Código SystemVerilog (en repositorio)

- Procesador VLIW propio (definido por grupo en el Proyecto Grupal I) extendido con jerarquía de memoria caché
- Modelo realista de memoria principal con latencias multi-ciclo y dominios de reloj separados
- ROM de instrucciones y jerarquía de caché de datos (L1-D y L2) con políticas configurables elegidas por el grupo, incluyendo su política de inclusión
- Write buffer (L1-D→L2) y eviction buffer (L2→memoria principal), ambos obligatorios
- Contadores de métricas de rendimiento integrados (hit/miss rates, IPC, stalls, invalidaciones hacia atrás si aplica)
- Testbenches comprensivos para validación de cada componente (procesador, caché, write buffers, memoria)
- Scripts de simulación automatizada con Icarus Verilog
- Makefile para compilación y ejecución de simulaciones

8.1.2. Documentación de Diseño (en repositorio)

La documentación debe incluirse en el directorio `docs/` del repositorio en formato markdown:

1. **Arquitectura de jerarquía de memoria:** Decisiones de diseño de caché con justificación técnica de políticas de escritura, reemplazo, **inclusión** (inclusiva o NINE, y su justificación dado que el proyecto es de un solo núcleo), y parámetros de los write buffers obligatorios (tamaño y mecanismo de RAW en memoria). Diagramas de la jerarquía L1/L2 y su integración con el pipeline.

2. **Organización/Microarquitectura:** Diagramas de bloques del sistema extendido mostrando pipeline, cachés, write/eviction buffers, controlador de memoria, y contadores de métricas. Explicación del flujo de datos e instrucciones.

3. **Análisis de rendimiento y validación:** Metodología de medición, benchmarks utilizados, y análisis de resultados. Comparación entre diferentes configuraciones y validación experimental.

8.2. Entregables - CE1108 Compiladores e Intérpretes

Los entregables para CE1108 se organizan en la documentación escrita del proyecto y los artefactos de software que la acompañan.

8.2.1. Documentación

- Introducción: Teoría necesaria, breve descripción del proyecto y qué es lo que se espera en el escrito.

- Ambiente de desarrollo: Todos los detalles de implementación y herramientas utilizadas durante el desarrollo del proyecto.

- Atributos: En esta sección se debe colocar la pregunta y respuesta para cada uno de los siguientes atributos:

    1. **Atributo de Conocimiento de Ingeniería (CI):**
        - ¿Cuáles conceptos matemáticos o de ciencias naturales y fundamentos especializados utilizó en la solución del proyecto? Explique con detalle y muestre un ejemplo.
    2. **Persona Ingeniera y el Mundo (PM):**
        - Identifique los aspectos sociales, económicos, legales, ambientales, de seguridad y salud en el análisis y evaluación del impacto ambiental para el desarrollo sostenible al resolver problemas complejos de ingeniería, específicamente en la parte de compiladores. (Recuerde que parte de la eficiencia de los compiladores es que se ejecuten menos instrucciones y por ende el hardware consuma menos potencia).
        - Explique la importancia de los aspectos sociales, económicos, legales, ambientales, de seguridad y salud en el análisis y evaluación del impacto ambiental para el desarrollo sostenible al resolver problemas complejos de ingeniería.
        - Explique cómo integrar los aspectos sociales, económicos, legales, ambientales, de seguridad y salud en el análisis y evaluación del impacto ambiental para el desarrollo sostenible al resolver problemas complejos de ingeniería.

- Detalles del diseño del programa desarrollado, tanto del software como del hardware (en caso de que aplique): Diagramas UML (obligatorio), diagrama de arquitectura (obligatorio), diagrama de funcionalidades (obligatorio), imágenes, descripciones, entre otros, todo lo que sea necesario para entender de mejor manera el diseño y funcionamiento del proyecto.

- Instrucciones de cómo se utiliza el proyecto.
- Tabla de actividades por cada estudiante: bitácora con el total de horas trabajadas.
- Conclusiones.
- Sugerencias y recomendaciones.
- Referencias.

8.2.2. Entregables

1. Archivos fuente.
2. Set de pruebas requeridos.
3. Documentación solicitada (junto con los atributos).

8.3. Presentación Final Conjunta

La defensa final será realizada de forma conjunta, demostrando la integración entre arquitectura y compilador:

- Demostración funcional del sistema completo (procesador + compilador + benchmarks)
- Análisis del impacto de optimizaciones de compilador en métricas de arquitectura
- Justificación de decisiones de diseño arquitectural y de optimización
- Q&A técnica cubriendo ambas perspectivas (hardware y software)

9. Evaluación

Este proyecto será evaluado de forma independiente por cada curso participante, utilizando criterios específicos alineados con los objetivos de aprendizaje de cada materia.

9.1. Evaluación - CE4301 Arquitectura de Computadores I

La evaluación se compone de:

1. Presentación proyecto funcional 75 %
2. Repositorio y documentación de diseño 25 %

9.1.1. Presentación proyecto funcional (75 %)

La defensa se realizará de **forma presencial** en un horario a definir. La defensa debe realizarse en un espacio de 25 minutos aproximadamente. Deberá mostrarse el sistema en funcionamiento, así como la compilación sin errores, simulación exitosa con Icarus Verilog, ejecución correcta del pipeline con jerarquía de caché y validación de métricas de rendimiento.
Los entregables que se revisarán **durante** la defensa son los siguientes:

1. **Arquitectura del procesador extendido:**
   a) Extensión del procesador VLIW propio (Proyecto Grupal I) con la jerarquía de memoria caché
   b) Justificación de decisiones de diseño de la jerarquía de caché (políticas de escritura, reemplazo, inclusión y parámetros de los write buffers)
   c) Integración correcta entre pipeline y sistema de memoria
2. **Implementación de jerarquía de caché:**
   a) Funcionamiento de caché L1-D con políticas elegidas
   b) Funcionamiento de caché L2 unificado
   c) Coherencia entre niveles de caché, incluyendo la política de inclusión elegida (invalidación hacia atrás si es inclusiva)
   d) Funcionamiento correcto del write buffer y del eviction buffer obligatorios, incluyendo el mecanismo elegido para el riesgo RAW en memoria
   e) Manejo correcto de stalls de pipeline por misses
3. **Métricas y análisis de rendimiento:**
   a) Contadores precisos de hit/miss rates, stalls, IPC
   b) Medición de impacto de diferentes configuraciones de caché
   c) Análisis comparativo entre políticas implementadas
   d) Validación de resultados con diferentes benchmarks

9.1.2. Repositorio y documentación de diseño (25 %)

La documentación del diseño deberá contener las siguientes secciones:

- **Gestión de repositorio (10 %):**
  - **Uso adecuado de Git con flujo de trabajo profesional (5 %):** El proyecto debe mantener un repositorio Git con historial de commits descriptivos, uso de ramas de desarrollo para nuevas funcionalidades. Se evaluará la calidad de los mensajes de commit, la organización de branches, y el uso de pull requests.
  - **README completo y claro (2 %):** El repositorio debe incluir un archivo README detallado que explique el propósito del proyecto, instrucciones de compilación y ejecución con Icarus Verilog, dependencias necesarias, estructura del código, y ejemplos de uso con benchmarks.

- **Gestión de issues y milestones (3 %):** Demostrar el uso de issues para tracking de funcionalidades, bugs y mejoras, organizados en milestones que reflejen el progreso del proyecto.

- **Documentación de diseño (15 %):** La documentación del diseño deberá agregarse al repositorio, en un directorio *docs*. La documentación deberá presentarse en formato markdown (.md), y deberá contener las siguientes secciones:

  1. **Arquitectura de jerarquía de memoria (5 %):** Descripción detallada de las decisiones de diseño de caché, incluyendo políticas de escritura, reemplazo, **inclusión** (con justificación, dado que el proyecto no tiene coherencia multinúcleo), y parámetros de los write buffers obligatorios. Diagramas de la jerarquía L1/L2 y su integración con el pipeline del procesador.
  2. **Organización/Microarquitectura (5 %):** Descripción de la organización interna del sistema extendido en SystemVerilog, incluyendo diagramas de bloques que muestren los componentes principales (pipeline, cachés, write/eviction buffers, controladores, contadores de métricas) y sus interconexiones. Explicación del flujo de datos e instrucciones.
  3. **Análisis de rendimiento y validación (5 %):** Descripción de la metodología de medición implementada, incluyendo detalles sobre los contadores de rendimiento, benchmarks utilizados, y análisis de resultados. Comparación entre diferentes configuraciones y justificación de decisiones de diseño.

9.2. Evaluación - CE1108 Compiladores e Intérpretes

La evaluación se compone de:

1. Optimizaciones de compilador (Usando VLIW) 45 %
   a) Loop unrolling
   b) Spilling register
   c) Reordenamiento
   d) Eliminación de código muerto
2. IDE 15 %
3. Representación intermedia 15 %
4. Conjunto de pruebas 10 %
5. Documentación técnica 15 %

10. Entrega

La entrega de ambos componentes del proyecto debe realizarse a más tardar el 20 de noviembre a las 23:55 por TEC Digital.

- **CE4301 (Arquitectura de Computadores I):** La entrega se realiza mediante un archivo de texto (Word, PDF, TXT, etc.) subido a TEC Digital, que debe contener únicamente el enlace al repositorio del proyecto y el hash del último commit al momento de la entrega. El contenido evaluable (código, testbenches y documentación de diseño) debe estar en dicho commit del repositorio, no en el archivo subido a TEC Digital.
- **CE1108 (Compiladores e Intérpretes):** Los archivos fuente y la documentación solicitada se entregan directamente por TEC Digital.